

**Doc No:** P1438R1  
**Date:** 2019-03-10  
**Author:** Bill Seymour  
**Reply to:** stdbill.h@pobox.com  
**Audience:** SG6 (Numerics)

# A Rational Number Library for C++

**Bill Seymour**  
**2019-03-10**

---

## Introduction:

This paper proposes a rational number library for a C++ Technical Specification (TS).

Because almost every operation on rational numbers requires multiplication behind the scenes, numerators and denominators can become quite large in a hurry, easily overflowing any fundamental integer type; so this paper requires that the numerator and denominator be stored internally in an unlimited precision integer such as the one suggested by Pete Becker in [N4038](#).

---

## Revision history:

- [N3363](#) – post-Kona (original)
- [N3414](#) – pre-Portland
- [N3489](#) – post-Portland
- [N3611](#) – pre-Bristol
- [P1438R0](#) – pre-Kona
- P1438R1 – this paper

Many thanks to Daniel Krüglér for several excellent suggestions to get from N3363 to N3414.

Thanks also to Lawrence Crowl for his excellent suggestions to get to P1438R0.

The rounding mode argument to the [nearest](#) function now has the enum class rounding type in Lawrence Crowl's [P0105R1](#).

As suggested by SG-6 and LEWG-I at the February 2019 meeting in Kona,

- the `normalize()` member function will not modify `*this`, but rather will return a copy with the fraction reduced to lowest terms.
- the `num()` and `denom()` observers will not automatically call `normalize()` so that users who don't need the normalization don't have to pay for it.
- remove the I/O manipulators. (They might come back later, but they're not for version 1.)

Not yet reflected in this paper is that there should be a member function that returns the amount of memory that has been allocated for the numerator and denominator. The [integer](#) used internally has a `size()` member function, but it returns a number of decimal digits, not a number of bytes. LEWG-I suggests that, if this member function is added, it should not be called `size()`.

Also still to be added is a hashing mechanism that works on normalized values (because we want 1/2 and 2/4 to have the same hash).

The [seminumeric](#) namespace is a placeholder for whatever namespace will appear in the TS.

---

## Questions:

1. Do we want conversion from fixed- and/or floating-point decimals? Fixed-point binary? I think all of them; but it's not clear yet what will come out of SG-6 in this regard.
  2. Do we want rational literals? I think not. The rationale in Pete Becker's Open issue 7 in [N4038](#), that they wouldn't be `constexpr`, doesn't really apply because we can now do `constexpr` memory allocation, but we're not sure that we want to go through the spelling bikeshedding.
  3. This paper depends on there being an unlimited precision integer, but [N4038](#) seems to have been abandoned. Will anyone champion N4038?
- 

## [rational.math] Rational math

The header `<rational>` defines the `rational` class and several free functions and operators for doing arithmetic with rational numbers.

### Header `<rational>` synopsis

```
namespace std {
namespace experimental {
namespace seminumeric {

class rational;

rational operator+(const rational& val);
rational operator-(const rational& val);

rational operator+(const rational& lhs, const rational& rhs);
rational operator-(const rational& lhs, const rational& rhs);
rational operator*(const rational& lhs, const rational& rhs);
rational operator/(const rational& lhs, const rational& rhs);

rational operator+(const rational& lhs, const integer& rhs);
rational operator-(const rational& lhs, const integer& rhs);
rational operator*(const rational& lhs, const integer& rhs);
rational operator/(const rational& lhs, const integer& rhs);

rational operator+(const integer& lhs, const rational& rhs);
rational operator-(const integer& lhs, const rational& rhs);
rational operator*(const integer& lhs, const rational& rhs);
rational operator/(const integer& lhs, const rational& rhs);

bool operator==(const rational& lhs, const rational& rhs);
bool operator!=(const rational& lhs, const rational& rhs);
bool operator< (const rational& lhs, const rational& rhs);
bool operator> (const rational& lhs, const rational& rhs);
bool operator<=(const rational& lhs, const rational& rhs);
bool operator>=(const rational& lhs, const rational& rhs);
```

```

bool operator==(const rational& lhs, const integer& rhs);
bool operator!=(const rational& lhs, const integer& rhs);
bool operator< (const rational& lhs, const integer& rhs);
bool operator> (const rational& lhs, const integer& rhs);
bool operator<=(const rational& lhs, const integer& rhs);
bool operator>=(const rational& lhs, const integer& rhs);

bool operator==(const integer& lhs, const rational& rhs);
bool operator!=(const integer& lhs, const rational& rhs);
bool operator< (const integer& lhs, const rational& rhs);
bool operator> (const integer& lhs, const rational& rhs);
bool operator<=(const integer& lhs, const rational& rhs);
bool operator>=(const integer& lhs, const rational& rhs);

void swap(rational& lhs, rational& rhs) noexcept;

integer floor(const rational& val);
integer ceil (const rational& val);
integer trunc(const rational& val);
integer nearest(const rational& val, rounding mode = rounding::tie_to_even);
integer round(const rational& val);

rational abs(const rational& val);

rational reciprocal(const rational& val);

rational pow(const rational& val, const integer& exp);

template<class intT>
    rational modf(const rational& val, intT* intptr);

rational modf(const rational& val);

template<class charT, class traits>
    std::basic_istream<charT,traits>&
        operator>>(std::basic_istream<charT,traits>& lhs, rational& rhs);

template<class charT, class traits>
    std::basic_ostream<charT,traits>&
        operator<<(std::basic_ostream<charT,traits>& lhs, const rational& rhs);

} // namespace seminumeric
} // namespace experimental
} // namespace std

```

## **[rational.class] Class rational**

```

class rational {
public:
    rational() noexcept;

    rational(const rational& rat);
    rational(rational&& rat) noexcept;

    explicit rational(float num);
    explicit rational(double num);
    explicit rational(long double num);

    explicit rational(integer num);

    rational(integer num, integer den);

    ~rational() noexcept;

```

```

rational& operator=(const rational& rat);
rational& operator=(rational&& rat) noexcept;

rational& operator=(integer num);
rational& assign(integer num, integer den);

void swap(rational& rhs) noexcept;

rational normalize() const;

integer numer() const;
integer denom() const;

explicit operator bool() const noexcept;

rational& negate() noexcept;
rational& invert() noexcept;

rational& operator++();
rational& operator--();

rational operator++(int);
rational operator--(int);

rational& operator+=(const integer& rhs);
rational& operator-=(const integer& rhs);
rational& operator*=(const integer& rhs);
rational& operator/=(const integer& rhs);

rational& operator+=(const rational& rhs);
rational& operator-=(const rational& rhs);
rational& operator*=(const rational& rhs);
rational& operator/=(const rational& rhs);
};

```

The numerator and denominator shall be stored internally in a [`std::experimental::seminumeric::integer`](#).

### **[rational.members] rational member functions:**

```
rational() noexcept;
```

*Effects:* Constructs a rational with a numerator equal to zero and a denominator equal to one.

```
rational(const rational& rat);
rational(rational&& rat);
```

*Effects:* Constructs a rational with a value of rat.

```
explicit rational(integer num);
```

*Effects:* Constructs a rational with a value of num.

```
explicit rational(float val);
explicit rational(double val);
explicit rational(long double val);
```

*Effects:* Constructs a rational with a value equal to val.

```
rational(integer num, integer den);
```

*Requires:* den != 0

*Effects:* Constructs a rational given the specified numerator and denominator.

`~rational() noexcept;`

*Effects:* Destructs `*this`.

`rational& operator=(const rational& rhs);`  
`rational& operator=(rational&& rhs) noexcept;`

*Effects:* Assigns `rhs` to `*this`.

*Returns:* `*this`.

`rational& operator=(integer num);`

*Effects:* Assigns `num` to `*this`.

*Returns:* `*this`.

`rational& assign(integer num, integer den);`

*Requires:* `den != 0`

*Effects:* Assigns the specified numerator and denominator to `*this`.

*Returns:* `*this`.

`void swap(rational& rhs) noexcept;`

*Effects:* Swaps `*this` and `rhs`.

`rational normalize() const;`

*Returns:* a rational equal to `*this`, but with the numerator and denominator having no common factor other than 1 and the denominator greater than 0. If the numerator is 0, the denominator shall be 1.

`integer numer() const;`

*Returns:* the (possibly not normalized) numerator by value.

`integer denom() const;`

*Returns:* the (possibly not normalized) denominator by value.

`explicit operator bool() const noexcept;`

*Returns:* as if `*this != 0`.

`rational& negate() noexcept;`

*Effects:* changes the sign of `*this`.

*Returns:* `*this`.

`rational& invert() noexcept;`

*Effects:* swaps the numerator and denominator.

*Requires:* the numerator is non-zero.

*Returns:* \*this.

## **[rational.member.ops] rational member operators:**

`rational& operator++();`

*Effects:* adds 1 to \*this and stores the result in \*this.

*Returns:* \*this.

`rational& operator--();`

*Effects:* subtracts 1 from \*this and stores the result in \*this.

*Returns:* \*this.

`rational operator++(int);`

*Effects:* adds 1 to \*this and stores the result in \*this.

*Returns:* the value of \*this before the addition.

`rational operator--(int);`

*Effects:* subtracts 1 from \*this and stores the result in \*this.

*Returns:* the value of \*this before the subtraction.

`rational& operator+=(const integer& rhs);`

*Effects:* adds the integer value rhs to \*this and stores the result in \*this.

*Returns:* \*this.

`rational& operator-=(const integer& rhs);`

*Effects:* subtracts the integer value rhs from \*this and stores the result in \*this.

*Returns:* \*this.

`rational& operator*=(const integer& rhs);`

*Effects:* multiplies \*this by the integer value rhs and stores the result in \*this.

*Returns:* \*this.

`rational& operator/=(const integer& rhs);`

*Effects:* divides \*this by the integer value rhs and stores the result in \*this.

*Requires:* rhs != 0.

*Returns:* \*this.

`rational& operator+=(const rational& rhs);`

*Effects:* adds the rational value rhs to \*this and stores the result in \*this.

*Returns:* \*this.

```
rational& operator--(const rational& rhs);
```

*Effects:* subtracts the rational value rhs from \*this and stores the result in \*this.

*Returns:* \*this.

```
rational& operator*=(const rational& rhs);
```

*Effects:* multiplies \*this by the rational value rhs and stores the result in \*this.

*Returns:* \*this.

```
rational& operator/=(const rational& rhs);
```

*Effects:* divides \*this by the rational value rhs and stores the result in \*this.

*Requires:* rhs != 0.

*Returns:* \*this.

## **[rational.ops] rational non-member operations:**

```
rational operator+(const rational& val);
```

*Returns:* rational(val).

```
rational operator-(const rational& val);
```

*Returns:* rational(val).negate().

```
rational operator+(const rational& lhs, const rational& rhs);
```

*Returns:* rational(lhs) += rhs.

```
rational operator-(const rational& lhs, const rational& rhs);
```

*Returns:* rational(lhs) -= rhs.

```
rational operator*(const rational& lhs, const rational& rhs);
```

*Returns:* rational(lhs) \*= rhs.

```
rational operator/(const rational& lhs, const rational& rhs);
```

*Requires:* rhs != 0.

*Returns:* rational(lhs) /= rhs.

```
rational operator+(const rational& lhs, const integer& rhs);
```

*Returns:* rational(lhs) += rhs.

```
rational operator-(const rational& lhs, const integer& rhs);
```

*Returns:* rational(lhs) -= rhs.

`rational operator*(const rational& lhs, const integer& rhs);`

*Returns:* `rational(lhs) *= rhs.`

`rational operator/(const rational& lhs, const integer& rhs);`

*Requires:* `rhs != 0.`

*Returns:* `rational(lhs) /= rhs.`

`rational operator+(const integer& lhs, const rational& rhs);`

*Returns:* `rational(rhs) += lhs.`

`rational operator-(const integer& lhs, const rational& rhs);`

*Returns:* `rational(rhs).negate() += lhs.`

`rational operator*(const integer& lhs, const rational& rhs);`

*Returns:* `rational(rhs) *= lhs.`

`rational operator/(const integer& lhs, const rational& rhs);`

*Requires:* `rhs != 0.`

*Returns:* `rational(rhs).invert() *= lhs.`

`bool operator==(const rational& lhs, const rational& rhs);`

*Returns:* as if `lhs.numer() * rhs.denom() == rhs.numer() * lhs.denom().`

`bool operator!=(const rational& lhs, const rational& rhs);`

*Returns:* `!(lhs == rhs).`

`bool operator<(const rational& lhs, const rational& rhs);`

*Returns:* as if `lhs.numer() * rhs.denom() < rhs.numer() * lhs.denom().`

`bool operator>(const rational& lhs, const rational& rhs);`

*Returns:* `rhs < lhs.`

`bool operator<=(const rational& lhs, const rational& rhs);`

*Returns:* `lhs < rhs || lhs == rhs.`

`bool operator>=(const rational& lhs, const rational& rhs);`

*Returns:* `lhs > rhs || lhs == rhs.`

`bool operator==(const rational& lhs, const integer& rhs);`

*Returns:* as if after normalization `lhs.numer() == rhs && lhs.denom() == 1.`

`bool operator!=(const rational& lhs, const integer& rhs);`

*Returns:* `!(lhs == rhs).`



```
bool operator<(const rational& lhs, const integer& rhs);
```

*Returns:* as if `lhs.numer() < rhs * lhs.denom()`.

```
bool operator>(const rational& lhs, const integer& rhs);
```

*Returns:* as if `lhs.numer() > rhs * lhs.denom()`.

```
bool operator<=(const rational& lhs, const integer& rhs);
```

*Returns:* `lhs < rhs || lhs == rhs`.

```
bool operator>=(const rational& lhs, const integer& rhs);
```

*Returns:* `lhs > rhs || lhs == rhs`.

```
bool operator==(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs == lhs`.

```
bool operator!=(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs != lhs`.

```
bool operator<(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs > lhs`.

```
bool operator>(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs < lhs`.

```
bool operator<=(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs >= lhs`.

```
bool operator>=(const integer& lhs, const rational& rhs);
```

*Returns:* `rhs <= lhs`.

## **[rational.conv] rational numeric conversions:**

```
integer floor(const rational& val);
```

*Returns:* the most positive integer not greater than `val`.

```
integer ceil(const rational& val);
```

*Returns:* the most negative integer not less than `val`.

```
integer trunc(const rational& val);
```

*Returns:* the integer part of `val` truncated toward zero.

```
integer nearest(const rational& val, rounding mode = rounding::tie_to_even);
```

*Returns:* the integer nearest in value to `val`. In the case of a tie (if `val.denom()` would be 2), the returned value shall be rounded as specified by `mode`.

```
integer round(const rational& val);
```

*Returns:* nearest(val, [rounding](#)::all\_away\_zero)

### **[rational.swap] rational swap:**

```
void swap(rational& lhs, rational& rhs) noexcept;
```

*Effects:* lhs.swap(rhs).

### **[rational.io] I/O operators that apply to rational objects:**

```
template<class charT, class traits>
std::basic_istream<charT, traits>&
operator>>(std::basic_istream<charT, traits>& lhs, rational& rhs);
```

The extraction operator reads an integer value which it takes to be a numerator, and if that is immediately followed by a division sign, reads another integer value which it takes to be a denominator. If no division sign immediately follows the numerator, the denominator shall be assumed to be 1.

If the operator reads a value of zero when expecting a denominator, it shall set the stream's failbit. If an `ios_base::failure` exception is thrown, or if the stream is not good() when the operator returns, rhs shall be untouched.

The stream's locale and all its format flags except skipws shall apply separately to the numerator and the denominator. skipws shall apply to the numerator only.[footnote]

[footnote]Thus, if a denominator is present, the division sign must immediately follow the numerator, and the denominator must immediately follow the division sign, without any intervening characters including whitespace.

```
template<class charT, class traits>
std::basic_ostream<charT, traits>&
operator<<(std::basic_ostream<charT, traits>& lhs, const rational& rhs);
```

The insertion operator shall write a rational to the specified output stream; and the numerator and denominator shall be written in whatever format is appropriate given the stream's flags and locale. The showpos and showbase flags shall affect the numerator only. The output shall be in lowest terms as if rhs.normalize() had been called just prior to doing the output.

### **[rational.over] Additional overloads:**

```
rational abs(const rational& val);
```

*Returns:* rational(val) if val >= 0; otherwise -val.

```
rational reciprocal(const rational& val);
```

*Returns:* rational(val).invert().

```
rational pow(const rational& val, const integer& exp);
```

*Requires:* val != 0 || exp >= 0.

*Returns:* rational(1) if exp == 0; otherwise val raised to the exp power.

*Remarks:* the exponent must be an integer since raising to a non-integer power yields an irrational value in general.

```
template<class intT>
    rational modf(const rational& val, intT* intptr);
```

*Effects:* if `intptr` is not a null pointer, `*intptr` shall be assigned `trunc(val)`.

*Returns:* `val - trunc(val)`.

```
rational modf(const rational& val);
```

*Returns:* `val - trunc(val)`.

---

All suggestions and corrections will be welcome; all flames will be amusing.

Bill Seymour, [stdbill.h@pobox.com](mailto:stdbill.h@pobox.com).